Reviewer Pack — Minimal Order of Formal Concept Analysis and Frege Proof Com…

Entry c863056b2246 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-06 16:04:49 UTC

Minimal Order of Formal Concept Analysis and Frege Proof Complexity

Entry ID: `c863056b2246` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-06-06 16:04:49 UTC

1. Conjecture

Field A (mathematical branch): Formal Concept Analysis (FCA) **Field B** (complexity object): Boolean Satisfiability (Frege Proof Complexity)

Statement:

> For every Boolean formula φ, the minimal order of formal concepts in its associated concept lattice is linearly correlated with its Frege proof complexity, such that the number of formal concepts |C(φ)| = Θ(w_F(φ)), where w_F(φ) denotes the Frege proof width of φ.

Rationale (proposer's reasoning):

> Formal Concept Analysis (FCA) provides a framework to analyze binary relations and their associated conceptual hierarchies. If this conjecture holds, it suggests that properties of the concept hierarchy can offer insights into the complexity of proving satisfiability using Frege-style proofs, which could lead to new methods for analyzing proof complexity.

Taxonomy category: `FCA_to_Boolean_Satisfiability` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `7d92dde4278c5972`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> For a Boolean formula φ, if |C(φ)|/w_F(φ) is within ±10% of 1 for all 30 seeds and the Pearson correlation coefficient is ≥ 0.9.

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM₁	LLM₂
RELATIVIZATION	SAFE	0.90	UNCERTAIN	SAFE

Barrier	Verdict	Confidence	LLM_1	LLM_2
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: `ADJACENT_OK` against 7 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - `formal concept analysis AND boolean satisfiability AND frege proof complexity` - `minimal order AND formal concepts AND theta w_frege` - `concept lattice AND Frege proof width AND FCA`

Top relevant hits considered: - [http://arxiv.org/abs/1210.2401v1] Distributed Formal Concept Analysis Algorithms Based on an Iterative MapReduce Framework - [http://arxiv.org/abs/1107.2822v1] A Survey on how Description Logic Ontologies Benefit from Formal Concept Analysis - [http://arxiv.org/abs/1809 The Origins Space Telescope (OST) Mission Concept Study Interim Report - [http://arxiv.org/abs/1904.01605v1] On the Formalization of Importance Measures using HOL Theorem Proving - [http://arxiv.org/abs/1405.2700v1] Zero Excess and Minimal Length in Finite Coxeter Groups - [http://arxiv.org/abs/1707.06967v1] Formal Analysis of Linear Control Systems using Theorem Proving - [http://arxiv.org/abs/1411.1128v1] Non-renormalization theorem and cyclic Leibniz rule in lattice supersymmetry

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import random
from fractions import Fraction
from itertools import combinations

def generate_formula(n):
    variables = [f'x{i}' for i in range(1, n+1)]
    clauses = []
    for _ in range(n):
        clause = random.sample(variables, 2)
        if random.choice([True, False]):
            clause = [f'not {v}' for v in clause]
        clauses.append(' or '.join(clause))
    return ' and '.join(clauses)

def evaluate_formula(formula, assignment):
    stack = []
    tokens = formula.split()
    i = 0
    while i < len(tokens):
        token = tokens[i]
        if token == 'not':
            next_token = tokens[i+1]
            if next_token.startswith('x'):
                stack.append(not assignment[next_token[1:]])
            else:
                stack.append(not evaluate_formula(next_token, assignment))
```

```
            i += 2
        elif token in ['and', 'or']:
            op = token
            right = stack.pop()
            left = stack.pop()
            if op == 'and':
                stack.append(left and right)
            else:
                stack.append(left or right)
            i += 1
        else:
            if token.startswith('x'):
                stack.append(assignment[token[1:]])
            else:
                stack.append(evaluate_formula(token, assignment))
            i += 1
    return stack.pop()

def find_minimal_order(formula):
    n = len(formula.split())
    variables = [f'x{i}' for i in range(1, n+1)]
    min_order = float('inf')
    for k in range(1, n+1):
        for assignment in combinations(variables, k):
            assignment_dict = {var: True if var in assignment else False for var in variables}
            if evaluate_formula(formula, assignment_dict) == 0:
                min_order = min(min_order, k)
    return min_order

def frege_proof_width(formula):
    # This is a placeholder function. Implement the actual Frege proof width calculation.
    # For simplicity, we assume it's proportional to the number of variables.
    n = len(formula.split())
    return n

def run_trial(seed: int) -> dict:
    random.seed(seed)
    results = []
    for _ in range(30):
        formula = generate_formula(random.randint(5, 40))
        min_order = find_minimal_order(formula)
        proof_width = frege_proof_width(formula)
        if proof_width == 0:
            return {
                "metric_name": "Min Order / Proof Width Ratio",
                "metric_value": None,
                "instances_tested": 1,
                "n_max": len(formula.split()),
                "conjecture_holds": False,
                "counterexample": f"Formula: {formula}, Min Order: {min_order}, Proof Width: {proo
            }
        ratio = Fraction(min_order, proof_width)
        results.append(ratio)
    mean_ratio = sum(results) / len(results)
    return {
        "metric_name": "Min Order / Proof Width Ratio",
```

```
        "metric_value": float(mean_ratio),
        "instances_tested": 30,
        "n_max": max(len(formula.split()) for formula in [generate_formula(random.randint(5, 40))
        "conjecture_holds": all(abs(ratio - mean_ratio) <= Fraction(10, 100) * abs(mean_ratio) for
        "counterexample": ""
    }

if __name__ == "__main__":
    seeds = [int(arg) for arg in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3
    results = []
    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
        results.append(trial_result)

    mean_value = sum(result["metric_value"] for result in results if result["metric_value"] is not
    std_value = (sum((result["metric_value"] - mean_value) ** 2 for result in results if result["m
    support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)
    if all(result["conjecture_holds"] for result in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_frac
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_frac
    else:
        counterexample = next(result["counterexample"] for result in results if not result["conjec
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conj
        print(f"RESULT: FALSIFIED counterexample=\"{counterexample}\" first_failing_seed={first_fa
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_9f84582a.py", line 106, in <module>
    trial_result = run_trial(seed)
                   ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_9f84582a.py", line 79, in run_trial
    min_order = find_minimal_order(formula)
                ^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_9f84582a.py", line 64, in find_minimal_
    if evaluate_formula(formula, assignment_dict) == 0:
       ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_9f84582a.py", line 36, in evaluate_form
    stack.append(not assignment[next_token[1:]])
                     ~~~~~~~~~~^^^^^^^^^^^^^^^^
KeyError: '30'
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: `INCONCLUSIVE`

Reasoning:

> The test code crashed during execution, which prevents us from verifying the conjecture's conditions. | next: Re-run the test with debugging enabled to identify the cause of the crash and ensure it completes successfully.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	12314
2	preregistration	ollama_remote	glm4:latest	0	0	14898
3	novelty	ollama_remote	glm4:latest	0	0	8402
4	novelty	ollama_remote	glm4:latest	0	0	11113
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	16956
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14804
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9422
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13808
9	judge	ollama_remote	glm4:latest	0	0	16950

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 118666 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/c863056b2246.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/c863056b2246.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/c863056b2246.tar.gz` (if generated)